

如何在运行框架中构建模型路由器

How to Build a Model Router in the Harness

LangChain · Sydney Runkle, Eugene Yurtsev · 2026-10-01 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。

核心要点

- 许多任务并不需要前沿级别的智能。在我们的实验中，相比基线，模型路由将每项编程任务的成本中位数降低了 64%，同时质量没有明显变化。
- 有效的模型路由器应当位于运行框架中，而不是通用网关中。选择正确的模型，需要领域与任务上下文。这些上下文已经由运行框架组织起来，而网关通常并不具备。
- 有效的路由器设计，以可观测性和评测为基础。它要求我们理解 Agent 的任务空间，研究模型表现，并明确成功标准。

前沿 LLM 的价格仍然高昂。随着 Agent 日益普及并大规模运行，这种成本将难以承受。好在，大多数 Agent 并不是每项任务都需要前沿级别的智能。模型实验室自己也这么说：Anthropic 的模型选择指南指出，“对于许多应用，从 Claude Haiku 4.5 这样的速度更快、成本效益更高的模型入手，可能是最优做法。”

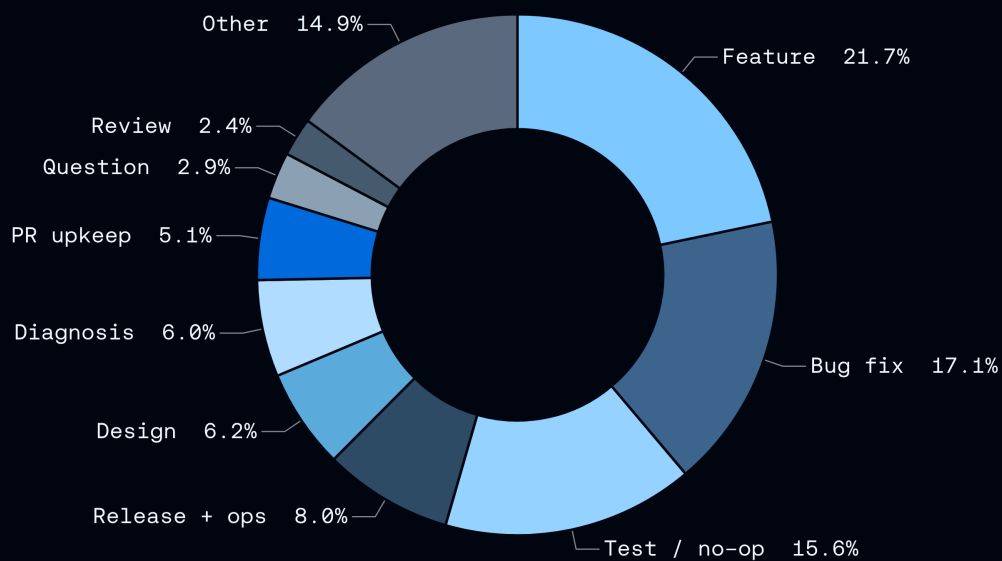
超过某个程度之后，就会出现收益递减：能力更强的模型带来的质量提升很少，成本和时延却持续上升。好的 Agent 需要实现模型、运行框架与任务之间的匹配：针对具体任务，提供正确的模型与正确的上下文。**模型路由器**为每项任务选择这样的模型。我们认为，这个路由决策应当属于 **Agent 运行框架**，而不是通用网关，因为选择正确模型所需的领域和任务上下文，恰恰是运行框架已经组织起来、而网关通常欠缺的内容。

最近，LangChain 每月在编程 Agent 上的开销开始快速上涨，我们切身感受到了这个问题。听到客户表达同样的担忧后，我们着手为开源编程 Agent Open SWE 构建有效的模型路由器。与此前始终使用顶级前沿模型的基线相比，它将每个会话的成本中位数降低了 64%，同时未测得质量变化。本文介绍我们如何构建这个路由器、从中学到了什么，以及你可以如何开始为自己的 Agent 加入模型路由。

第 1 步：理解任务

我们的试验平台是 Open SWE。工程师通过 Slack 和网页界面，用它询问代码库相关问题，并提出代码修改请求。在构建路由器之前，我们需要了解开发者会用 Open SWE 处理哪些类型的任务。我们从 LangSmith 执行轨迹中提取了会话级别的数据：收到的请求类型，以及每个会话的成本和轮数，后两者作为复杂度的近似指标。我们在 LangSmith Custom Apps 中开展探索，直接基于 Open SWE 的执行轨迹构建了一个小型界面。

我们选取了一周的交互会话，使用 LLM 分类器为每个会话标注任务类型。LangSmith Insights 也可以帮你执行轨迹进行这样的分组。代码修改占主导地位：新功能（22%）和缺陷修复（17%）是最大的两组，其次是测试或无实际操作的运行（16%）。分类是根据各会话的标题与元数据进行启发式判断的。



A week of Open SWE interactive threads by task type

我们还研究了不同任务类型对应的 Agent 执行轨迹特征有何差异。我们发现，与功能开发调研相关的会话往往更长，成本和轮数中位数更高。与测试和发布流程相关的会话，则通常相对较短、成本较低。

为了判断“复杂度”，我们同时使用总成本和调用次数作为信号：成本较直接地反映复杂度，因为更大的任务会使用更多 token；调用次数则更需要具体分析，因为调用次数多，既可能意味着任务更难，也可能意味着模型需要后续交互才能完成任务。



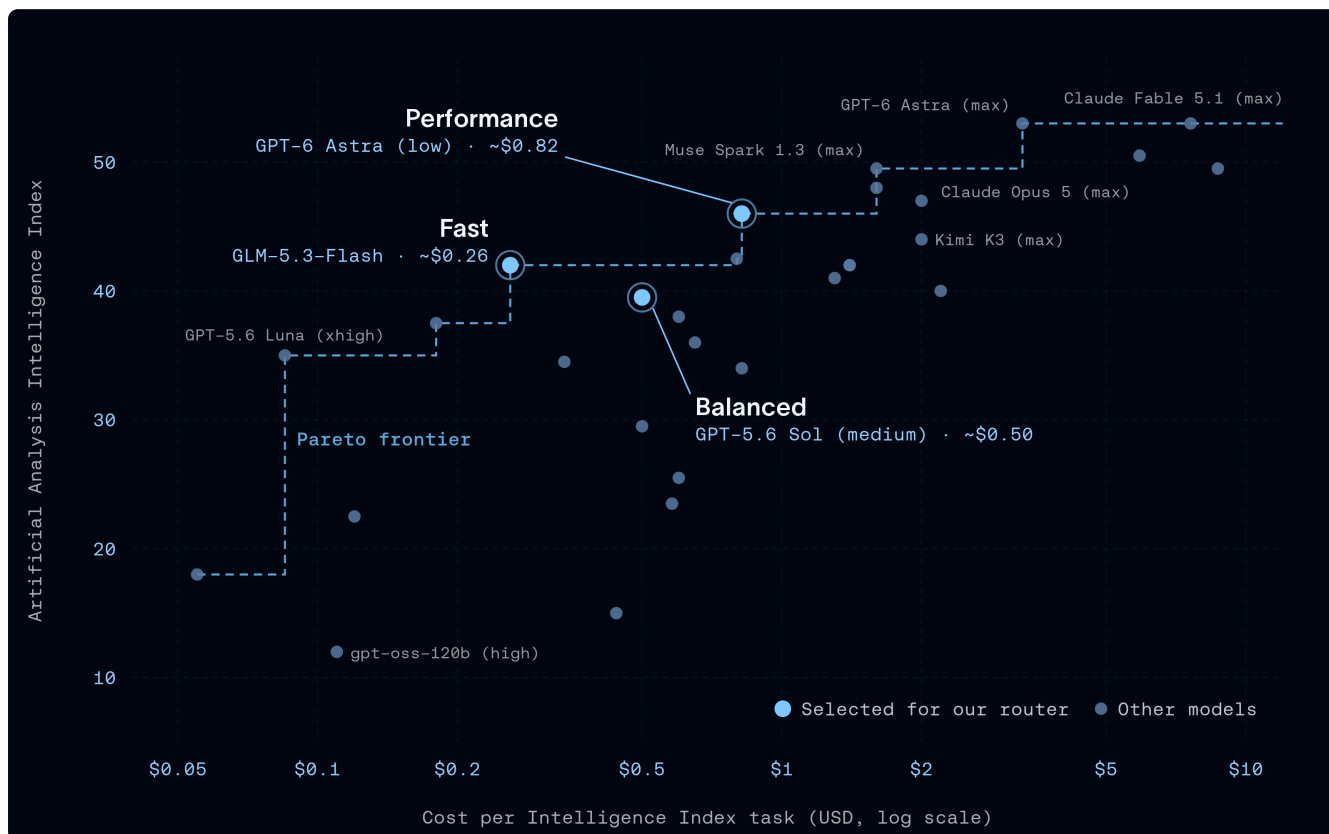
The six largest task categories (excluding "Other"), Aug 29 to Sep 5: thread count, median agent invocations, median LLM cost (threads with attributed usage), and complexity mix

收集数据时，Open SWE 的所有会话都交给一个顶级前沿模型处理。上述数据表明，考虑到任务复杂度的差异，Open SWE 处理的许多任务可能并不需要前高级别的智能。

这种任务复杂度上的差异，让我们提出了一个值得检验的假设：路由器可以从初始请求中推断任务类型和难度，将任务发送给更便宜或更快的模型，同时不降低最终结果的质量。

第 2 步：理解模型

Artificial Analysis 智能指数使用同一组任务对模型打分，并报告每项任务的成本，因此你可以把它们放到同一张智能水平与成本关系图上。帕累托前沿由那些在低成本与高智能之间处于最优权衡位置的模型构成。



Intelligence vs. cost per task, with the Pareto frontier, the three models we selected for our router, and a few other models for context. Chart data: Artificial Analysis Intelligence Index, as of Sep 9, 2026.

我们沿着这条曲线选定了三个模型，它们分别在成本、速度与智能之间取得不同的平衡：

- **快速档：**GLM-5.3-Flash (xhigh)
- **均衡档：**GPT-5.6 Sol (medium)
- **高性能档：**GPT-6 Astra (low)

我们选择了不同供应商的模型，其中快速档采用开放模型。GLM-5.3-Flash 与闭源模型一起位于帕累托前沿，这又一次表明，开放模型已经跨越了一个门槛。LangChain 不绑定特定模型，它提供通用的模型接口，在不同供应商之间都以相同方式工作。因此，当更好的模型出现时，只需修改路由器中的一行代码，就能将其替换进来。

第 3 步：在运行框架中构建路由器

在了解任务构成并选定三个档位之后，我们需要将任务与模型匹配：读取每个新请求，把它交给能够成功完成任务的最低成本档位。在 LangChain 中，这个决策天然适合放在中间件中。中间件可以替换 Agent 调用的模型，而无需改变 Agent 的其他任何部分，参见动态模型选择。

Open SWE 的路由器在收到会话中的第一条用户消息时运行。它有三个组成部分：

- **基础提示词**：告诉分类器它的工作是什么：选择可能完成任务的最低成本模型。
- **各档位的标准**：用简短、通俗的语言，描述每个档位应当承担什么工作。
- **分类模型**：读取请求，根据标准和提示词选择档位。

通用基准只是一个起点。各档位的标准应根据两类信息编写：你自己的任务分析，以及各供应商对其模型最擅长工作的介绍。我们结合了第 1 步的任务分类，以及 GPT-5.6 Sol、GPT-6 Astra 和 GLM-5.3-Flash 的供应商指南，编写基础提示词与各档位标准。

这些标准是针对 Open SWE 的任务集合编写的，因此路由器与 Open SWE 处理的任务深度耦合。**这正是它应当放在运行框架中的原因**：运行框架已经具备 Agent 特定任务所需的上下文，包括提示词、工具与领域知识，而通用网关并不具备。

第一版使用支持结构化输出的 LLM，并以用户请求作为提示。如今，分类器运行在新发布的决策模型 Jev 上，使分类速度提高了近 50 倍。具体做法请参阅《借助 Jev 构建运行框架》。

路由器在每个会话开始时只选择一次模型，整个会话都会使用该模型。一个自然的疑问是：如果会话中途改变了主题或复杂度，怎么办？这个简化版路由器尚未处理这一问题，但我们会在下方“下一步”一节讨论会话中途的路由。

第 4 步：跟踪任务结果

路由器的价值在于降低成本，但前提是质量不能退步。路由器必须做好两件事：所选模型能够完成任务，而且应当是能完成任务的模型中最便宜、最快的那一个。这意味着你需要一种跟踪任务结果的方法。有两种做法：

1. 离线评测让路由器在固定数据集上运行，以便安全、可重复地比较不同版本。难点在于数据集：它必须贴近真实流量，并按照用户关心的标准评分。对于编程 Agent，这意味着 PR 的质量与可评审性，而这些很难离线评分。
2. A/B 测试将实时会话分给路由器与单模型基线，并使用每个会话都能测量的成功指标进行比较。实时流量天然构成具有代表性的数据集，但缺点是用户可能会受到尚非最优的路由器影响。

我们运行了 A/B 测试，并使用两种结果信号：

- **合并的 PR**：Open SWE 现在会记录它创建的每个 PR，以及该 PR 最终被合并还是关闭。每个会话合并的 PR 数，成为我们的主要成功指标。
- **用户反馈**：我们给 Open SWE 加入了点赞和点踩功能，让用户可以评价任何会话，包括那些不会产生 PR 的问答会话。每条评价都会作为反馈记录到该会话的 LangSmith 执行轨迹上。

我们在一个 LangSmith Custom App 中跟踪这两类信号。合并的 PR 是更有力的信号。用户反馈比较稀疏，因为只有少部分会话会得到评价；不过，我们正是通过这些反馈了解到路由错误，例如下面第一次测试中引用的那些评论。

实验

第一次 A/B 测试，将路由器与始终使用最强模型的策略进行比较：一半会话经过路由器，另一半始终使用 GPT-6 Astra，总计 973 个会话。

Metric	Always Astra	Routed	Difference	p-value
Threads	477	496	–	–
Median cost per thread	\$2.61	\$0.94	–64%	–
Mean cost per thread	\$4.76	\$2.75	–42%	–
p90 cost per thread	\$11.07	\$6.93	–37%	–
Threads ending in a merged PR	27.3%	29.2%	+2.0 pp	0.49
PR open rate	39.6%	38.9%	–0.7 pp	0.82

Sep 16–22, 2026, 973 threads

没有测得质量上的变化。经过路由的会话中，29.2% 最终产生了被合并的 PR，对照组为 27.3% ($p = 0.49$)。PR 创建率也基本持平：38.9% 对 39.6%， $p = 0.82$ 。

成本大幅下降。路由组的会话成本中位数为 0.94 美元，对照组为 2.61 美元，下降了 64%。平均成本下降 42%，第 90 百分位成本下降 37%，说明节省并不只是由少数极低成本的异常值带来的。

多数请求并不需要最强模型。在经过路由的会话中，56% 进入均衡档，34% 进入快速档，只有 10% 进入高性能档。各档位之间的成本差距很大：会话成本中位数分别为快速档 0.097 美元、均衡档 1.50 美元、高性能档 2.88 美元，相差约 30 倍。



LLM cost per thread, Sep 16 to 22 (log scale): always GPT-6 Astra vs. routed, and routed threads split by tier. Tier violin widths scale with thread count. White lines and labels mark the median.

用户反馈也指向同样的结论。当一个简单请求由 GPT-6 Astra 处理时，工程师会直接指出成本过高，例如评论：“对这个查询来说太贵了”，以及“这个请求不该被路由到高性能模型”。

💡 考虑到对照组使用的是我们最昂贵的模型，这个结果并不令人意外。但许多 Agent 都会受益于这种改变：不少 Agent 过于偏重质量，最终花了过多的钱。对于任务类型多样的 Agent，路由尤其能够降低成本。

我们还做了一个相反的对照实验：一半会话经过路由器，另一半始终使用快速模型。这个测试在一天之内就被终止了，尚未产生具有统计意义的结果。工程师几乎立刻就指出了只用快速模型的实验组存在问题；较低的输出质量正在干扰他们的工作效率。

下一步

这个模型路由器实现是一项概念验证，为构建最优路由器提供了基础。以下是可以进一步探索的改进方向：

- 在 DeepSWE 或其他编程基准上测试路由器，这样我们就可以针对受控基线评测路由决策，而不必只依赖生产流量上的 A/B 测试。
- 为子 Agent 选择模型。目前，子 Agent 的模型选择独立于路由器。如果也对子 Agent 进行路由，尤其是在长时间任务中，可能进一步降低成本。

- **在会话中途重新路由。**当新消息与之前的内容差异足够大时，例如问答变成了缺陷修复，更换模型可能值得。代价在于提示词缓存：切换模型会丢弃缓存，因此新模型需要按完整价格重新读取会话。对于异步 Agent，这个成本往往可以忽略，因为当缓存有效期较短（例如 5 分钟）时，它通常本来就会在两轮用户交互之间过期。
- **利用更多信号优化路由标准**，例如从执行轨迹中挖掘用户情绪：找出用户感到不满的会话，这可能意味着任务需要更强的模型。

如何开始

如果你要为自己的 Agent 增加路由，我们建议从以下几步开始：

1. **理解任务。**执行轨迹真实记录了用户要求 Agent 做什么，LangSmith Insights 则帮助你发现其中的模式，让你在选择档位之前先了解任务构成。
2. **理解模型。**参考当前的基准评测，沿着成本与智能水平曲线选取几个模型。LangChain 的模型接口适用于不同供应商，因此你可以随时更换模型，无须重新设计应用。
3. **在运行框架中构建路由器。**把路由视为上下文工程，它类似于经典的特征工程：根据 Agent 所处的领域，决定路由器需要看到哪些信息，才能对模型适配性做出最佳判断。
4. **跟踪任务结果。**在启用路由前，先建立衡量成功的方法：评测、在线评估器，或执行轨迹上的用户反馈。如果构建评测数据集的成本太高或难度太大，在实时流量上进行 A/B 测试也很有效。

随着新模型，包括开放权重模型，不断进入能力前沿，最适合某项任务的模型也在持续变化。由于 LangChain 不绑定特定模型，要获得这些进步带来的收益，只需替换一个档位的模型，无须重建 Agent。

要为自己的 Agent 增加路由，你可以使用我们新发布的模型路由中间件。向它提供基础提示词、模型档位，以及各档位的标准，它就会在每个会话开始时选择模型。欢迎通过 X 或 LangChain 的 GitHub issues 向我们反馈。

延伸阅读

- GitHub 上的 Open SWE
- 借助 Jev 构建运行框架
- 模型路由中间件文档
- LangSmith 可观测性文档
- LangSmith 评测文档
- LangSmith Custom Apps
- 在 LangSmith 中记录用户反馈
- Artificial Analysis 智能指数

致谢

感谢 Mason Daugherty、Kevin Frank 和 Harrison Chase 为本文提供的细致反馈。也感谢支持本次实验的 OpenSWE 团队！